IPL Best Batsman / Best Bowler Prediction

Goal

Run all cells in notebook

Build a machine learning pipeline to predict:

- the **best batsman** of each IPL match
- the **best bowler** of each IPL match

The project will use only methods covered in class and will later include a betting simulation based on model probabilities.

Current milestone

In this notebook section, I will:

1. load and inspect the IPL dataset
2. identify the important columns
3. check whether the data is ball-by-ball or already aggregated
4. create player-match level datasets for batsmen and bowlers

↳ 38 cells hidden

Create pre-match historical features

To make realistic predictions, features must be computed using only past matches.

Rolling averages will be used to capture recent player form.

↳ 8 cells hidden

Time-based train/test split

The dataset is split chronologically to avoid using future data during training.

```
train_batsman = batsman_model_df[batsman_model_df[DATE_COL] <= 2019]
test_batsman = batsman_model_df[batsman_model_df[DATE_COL] > 2019]

train_bowler = bowler_model_df[bowler_model_df[DATE_COL] <= 2019]
test_bowler = bowler_model_df[bowler_model_df[DATE_COL] > 2019]

print("Batsman:", train_batsman.shape, test_batsman.shape)
print("Bowler:", train_bowler.shape, test_bowler.shape)
```

```
Batsman: (9248, 21) (5596, 21)
Bowler: (7324, 18) (4291, 18)
```

Define feature sets

These columns will be used as model inputs for predicting the best batsman and best bowler.

```
batsman_features = [
    "runs_last_3",
    "runs_last_5",
    "sr_last_5",
    "fours_last_5",
    "sixes_last_5"
]

bowler_features = [
    "wickets_last_3",
    "wickets_last_5",
    "eco_last_5",
    "runs_conceded_last_5"
]
```

Clean feature matrices

Ensure all selected features are numeric and free from missing or infinite values before training.

```
def clean_feature_block(df, feature_cols):
    df = df.copy()
    df[feature_cols] = df[feature_cols].apply(pd.to_numeric, errors="coerce")
    df[feature_cols] = df[feature_cols].replace([np.inf, -np.inf], np.nan)
    df[feature_cols] = df[feature_cols].fillna(0)
    df[feature_cols] = df[feature_cols].clip(lower=-1e6, upper=1e6)
    return df

train_batsman = clean_feature_block(train_batsman, batsman_features)
test_batsman = clean_feature_block(test_batsman, batsman_features)

train_bowler = clean_feature_block(train_bowler, bowler_features)
test_bowler = clean_feature_block(test_bowler, bowler_features)
```

Build training and test matrices

Create the feature matrices `X` and target vectors `y` for both tasks.

```
X_train_bat = train_batsman[batsman_features].astype("float64")
y_train_bat = train_batsman["best_batsman_label"]

X_test_bat = test_batsman[batsman_features].astype("float64")
y_test_bat = test_batsman["best_batsman_label"]

X_train_bowl = train_bowler[bowler_features].astype("float64")
y_train_bowl = train_bowler["best_bowler_label"]

X_test_bowl = test_bowler[bowler_features].astype("float64")
y_test_bowl = test_bowler["best_bowler_label"]
```

Quick validation checks

Confirm that the feature matrices do not contain invalid values before training.

```
print("Batsman train inf:", np.isinf(X_train_bat.to_numpy()).sum())
print("Batsman test inf:", np.isinf(X_test_bat.to_numpy()).sum())
print("Bowler train inf:", np.isinf(X_train_bowl.to_numpy()).sum())
print("Bowler test inf:", np.isinf(X_test_bowl.to_numpy()).sum())

print("Batsman train nan:", np.isnan(X_train_bat.to_numpy()).sum())
print("Batsman test nan:", np.isnan(X_test_bat.to_numpy()).sum())
print("Bowler train nan:", np.isnan(X_train_bowl.to_numpy()).sum())
print("Bowler test nan:", np.isnan(X_test_bowl.to_numpy()).sum())
```

```
Batsman train inf: 0
Batsman test inf: 0
Bowler train inf: 0
Bowler test inf: 0
Batsman train nan: 0
Batsman test nan: 0
Bowler train nan: 0
Bowler test nan: 0
```

Train Random Forest models

A separate model is trained for batsman prediction and bowler prediction.

```
from sklearn.ensemble import RandomForestClassifier

bat_model = RandomForestClassifier(n_estimators=100, random_state=42)
bat_model.fit(X_train_bat, y_train_bat)

bowl_model = RandomForestClassifier(n_estimators=100, random_state=42)
bowl_model.fit(X_train_bowl, y_train_bowl)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Evaluate classification performance

Print the classification report for both models.

```
from sklearn.metrics import classification_report

print("BATSMAN MODEL")
print(classification_report(y_test_bat, bat_model.predict(X_test_bat)))

print("\nBOWLER MODEL")
print(classification_report(y_test_bowl, bowl_model.predict(X_test_bowl)))
```

Run all cells in notebook

```
BATSMAN MODEL
              precision    recall  f1-score   support

           0       0.93      1.00      0.96      5207
           1       0.00      0.00      0.00       389

    accuracy                           0.93      5596
   macro avg       0.47      0.50      0.48      5596
weighted avg       0.87      0.93      0.90      5596


BOWLER MODEL
              precision    recall  f1-score   support

           0       0.91      0.99      0.95      3911
           1       0.07      0.01      0.01       380

    accuracy                           0.91      4291
   macro avg       0.49      0.50      0.48      4291
weighted avg       0.84      0.91      0.87      4291
```

Predict match-level winners

The model outputs probabilities for each player.
For each match, the player with the highest probability is selected as the predicted best player.

```
test_batsman = test_batsman.copy()
test_bowler = test_bowler.copy()

test_batsman["pred_prob"] = bat_model.predict_proba(X_test_bat)[:, 1]
test_bowler["pred_prob"] = bowl_model.predict_proba(X_test_bowl)[:, 1]
```

```
pred_best_batsman = test_batsman.loc[
    test_batsman.groupby(MATCH_COL)["pred_prob"].idxmax()
].copy()

pred_best_bowler = test_bowler.loc[
    test_bowler.groupby(MATCH_COL)["pred_prob"].idxmax()
].copy()
```

Extract actual match winners

These rows represent the true best batsman and true best bowler in each test match.

```
actual_best_batsman = test_batsman[test_batsman["best_batsman_label"] == 1].copy()
actual_best_bowler = test_bowler[test_bowler["best_bowler_label"] == 1].copy()
```

Compare predicted and actual winners

Measure how often the model identifies the correct best player at the match level.

```
bat_compare = pred_best_batsman[[MATCH_COL, BATTER_COL, "pred_prob"]].merge(
    actual_best_batsman[[MATCH_COL, BATTER_COL]],
    on=MATCH_COL,
    suffixes=("_pred", "_actual")
)

bowl_compare = pred_best_bowler[[MATCH_COL, BOWLER_COL, "pred_prob"]].merge(
    actual_best_bowler[[MATCH_COL, BOWLER_COL]],
    on=MATCH_COL,
    suffixes=("_pred", "_actual")
)
```

```
bat_compare["correct"] = (bat_compare[f"{BATTER_COL}_pred"] == bat_compare[f"{BATTER_COL}_actual"]).astype(int)
bowl_compare["correct"] = (bowl_compare[f"{BOWLER_COL}_pred"] == bowl_compare[f"{BOWLER_COL}_actual"]).astype(int)

print("Match-level batsman accuracy:", bat_compare["correct"].mean())
print("Match-level bowler accuracy:", bowl_compare["correct"].mean())
```

Run all cells in notebook

```
Match-level batsman acc                    58354756
Match-level bowler accuracy: 0.08947368421052632
```

Inspect match-level predictions

Display a few examples of predicted versus actual winners.

```
bat_compare.head(10)
```

	match_id	batter_pred	pred_prob	batter_actual	correct
0	1216492	SA Yadav	0.320000	AT Rayudu	0
1	1216493	S Dhawan	0.290000	MA Agarwal	0
2	1216495	DA Warner	0.230000	DA Warner	1
3	1216496	SM Curran	0.456595	SV Samson	0
4	1216497	MP Stoinis	0.280000	N Rana	0
5	1216498	Mandeep Singh	0.200000	DA Warner	0
6	1216499	Q de Kock	0.240000	SA Yadav	0
7	1216500	PP Shaw	0.232000	SO Hetmyer	0
8	1216501	Shubman Gill	0.250000	RA Tripathi	0
9	1216502	DA Warner	0.200000	JO Holder	0

Next steps: Generate code with bat_compare New interactive sheet

```
bowl_compare.head(10)
```

	match_id	bowler_pred	pred_prob	bowler_actual	correct
0	1216492	DL Chahar	0.21	L Ngidi	0
1	1216493	K Rabada	0.12	K Rabada	1
2	1216494	CH Morris	0.16	Mohammed Siraj	0
3	1216495	Sandeep Sharma	0.30	Sandeep Sharma	1
4	1216496	L Ngidi	0.26	SM Curran	0
5	1216498	KK Ahmed	0.36	CJ Jordan	0
6	1216499	Mohammed Siraj	0.33	JJ Bumrah	0
7	1216500	R Ashwin	0.50	JC Archer	0
8	1216501	PJ Cummins	0.33	DJ Bravo	0
9	1216502	Rashid Khan	0.39	Sandeep Sharma	0

Next steps: Generate code with bowl_compare New interactive sheet

Betting simulation for batsman predictions

A bet is placed only when the model confidence exceeds a threshold.

```
stake = 10
win_return = 30
threshold = 0.60
```

```
bets_bat = bat_compare[bat_compare["pred_prob"] > threshold].copy()
bets_bat["profit"] = np.where(bets_bat["correct"] == 1, win_return - stake, -stake)

print("Total batsman bets:", len(bets_bat))
print("Batsman win rate:", bets_bat["correct"].mean() if len(bets_bat) > 0 else 0)
print("Batsman total profit:", bets_bat["profit"].sum())
```

```
Total batsman bets: 5
Batsman win rate: 0.0
Batsman total profit: -50
```

Betting simulation [illegible] predictions

Run all cells in notebook

The same rule is applied to bowler predictions.

```
bets_bowl = bowl_compare[bowl_compare["pred_prob"] > threshold].copy()
bets_bowl["profit"] = np.where(bets_bowl["correct"] == 1, win_return - stake, -stake)

print("Total bowler bets:", len(bets_bowl))
print("Bowler win rate:", bets_bowl["correct"].mean() if len(bets_bowl) > 0 else 0)
print("Bowler total profit:", bets_bowl["profit"].sum())
```

```
Total bowler bets: 6
Bowler win rate: 0.16666666666666666
Bowler total profit: -30
```

Combined betting summary

Summarize the betting performance across both tasks.

```
total_bets = len(bets_bat) + len(bets_bowl)
total_profit = bets_bat["profit"].sum() + bets_bowl["profit"].sum()

print("Combined total bets:", total_bets)
print("Combined total profit:", total_profit)
```

```
Combined total bets: 11
Combined total profit: -80
```

Save outputs

Save the match-level comparison tables and betting results for later analysis.

```
bat_compare.to_csv("/content/batsman_match_predictions.csv", index=False)
bowl_compare.to_csv("/content/bowler_match_predictions.csv", index=False)

bets_bat.to_csv("/content/batsman_bets.csv", index=False)
bets_bowl.to_csv("/content/bowler_bets.csv", index=False)
```